Week 8 Day 2 Lab 2

Apache NiFi: Beginner Exercises Guide

Objective:
Learn to build basic dataflows using Apache NiFi processors, understand core concepts, and develop practical data

Prerequisites: NiFi installed and running locally (default UI: https://localhost:8443/nifi or http://localhost:8080/nifi depending on your version), and basic familiarity with drag-and-drop in the canvas.

Exercise 1: Hello Flow

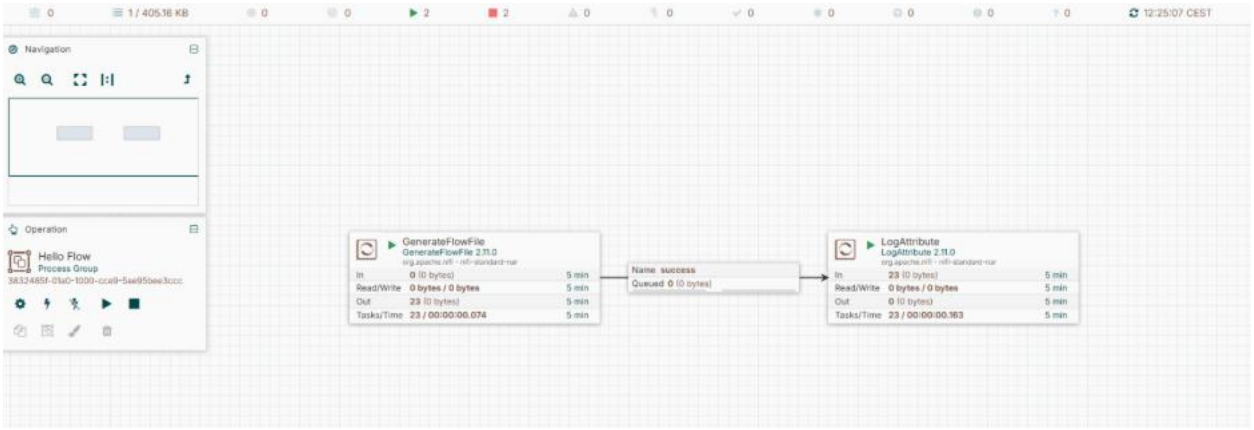
Goal: Understand FlowFiles, connections, and the Start/Stop lifecycle.

Processors used: `GenerateFlowFile`, `LogAttribute`

Steps

1. Drag a processor onto the canvas. Search for `GenerateFlowFile`, select it, click **Add**.
2. Double-click the processor to configure it. On the **Properties** tab, set:
    a. `File Size`: `0B`
    b. Leave other defaults.
3. Drag a second processor onto the canvas: `LogAttribute`. Add it.
4. Hover over `GenerateFlowFile` until an arrow appears, drag it onto `LogAttribute` to create a connection. In the dialog, check the `success` relationship, click **Add**.
5. Right-click `GenerateFlowFile` → **Configure** → **Scheduling** tab → set `Run Schedule` to `10 sec` (so it doesn't flood your log).
6. Select both processors (click, then shift-click) → right-click → **Start**.
7. Right-click `LogAttribute` → **View Data Provenance**. You should see events listed as FlowFiles pass through.

8. Open the NiFi app log (`logs/nifi-app.log` in your install directory) and confirm you see logged attribute output.
9. Stop both processors when done (select → right-click → **Stop**).



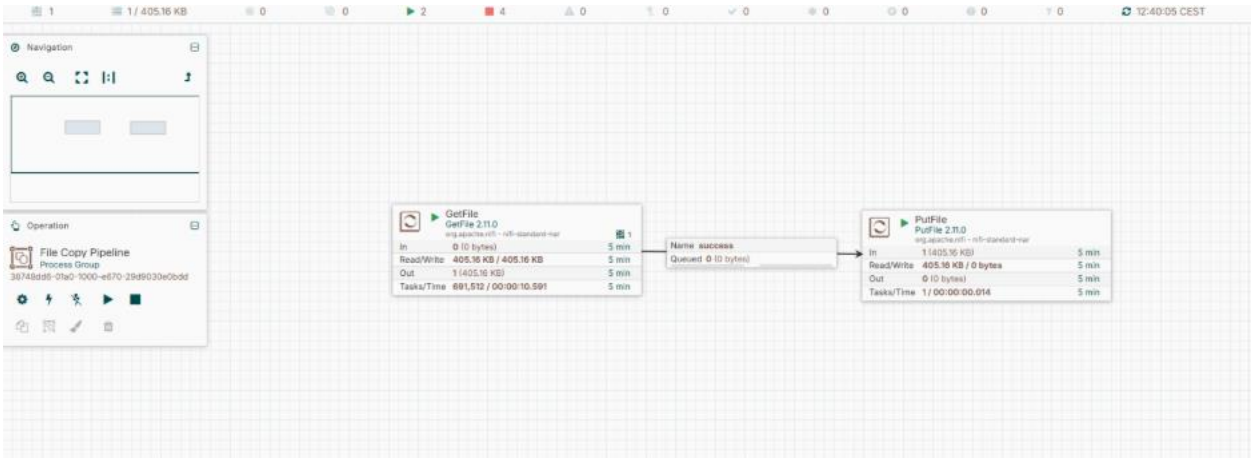
Exercise 2: File Copy Pipeline

Goal: Learn basic ingestion/egress processors and directory permissions.

Processors used: `GetFile`, `PutFile`

Steps

1. On your machine, create two folders: `~/nifi-input` and `~/nifi-output`.
2. Place a sample text file, e.g. `sample.txt`, inside `~/nifi-input`.
3. Add a `GetFile` processor. Configure:
    a. `Input Directory`: `/home/<you>/nifi-input`
    b. `Keep Source File`: `true` (so the file isn't deleted while you're testing)
4. Add a `PutFile` processor. Configure:
    a. `Directory`: `/home/<you>/nifi-output`
    b. `Conflict Resolution Strategy`: `replace`
5. Connect `GetFile` → `PutFile` on the `success` relationship.
6. Start both processors.
7. Check `~/nifi-output` — your file should appear there within a few seconds.



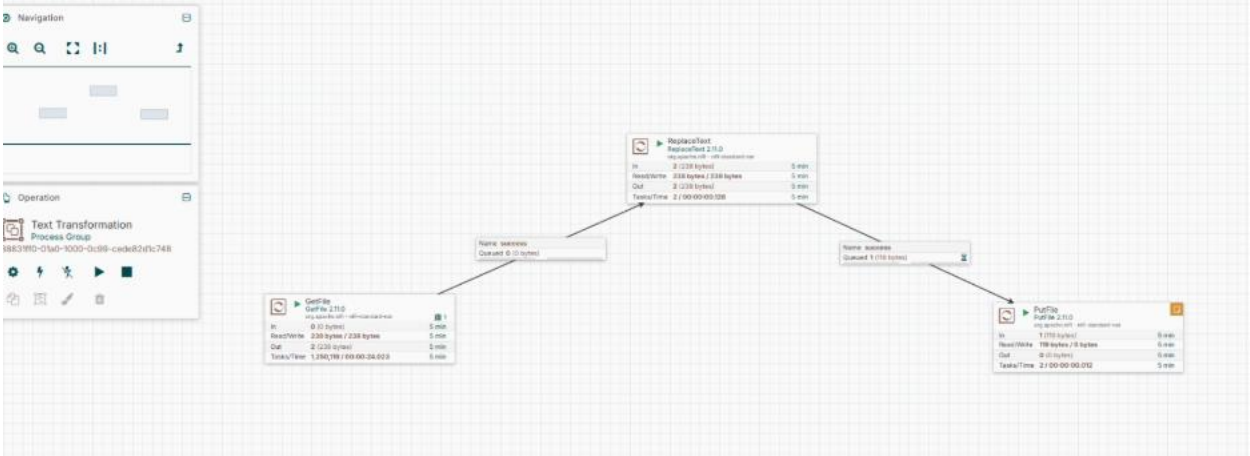
Exercise 3: Text Transformation

Goal: Practice configuring processor properties and regex-based replacement.

Processors used: `GetFile`, `ReplaceText`, `PutFile`

Steps

1. Reuse the `~/nifi-input` and `~/nifi-output` folders (or create new ones for this exercise).
2. Put a file in the input folder containing the word `hello` somewhere in its text.
3. Add `GetFile` (same config as Exercise 2).
4. Add `ReplaceText`. Configure:
    a. `Search Value`: `hello`
    b. `Replacement Value`: `HELLO`
    c. `Replacement Strategy`: `Regex Replace`
5. Add `PutFile` pointing to your output folder.
6. Connect `GetFile` → `ReplaceText` (`success`), then `ReplaceText` → `PutFile` (`success`).
7. Start all processors and confirm the output file now shows `HELLO` instead of `hello`.



Exercise 4: CSV to JSON

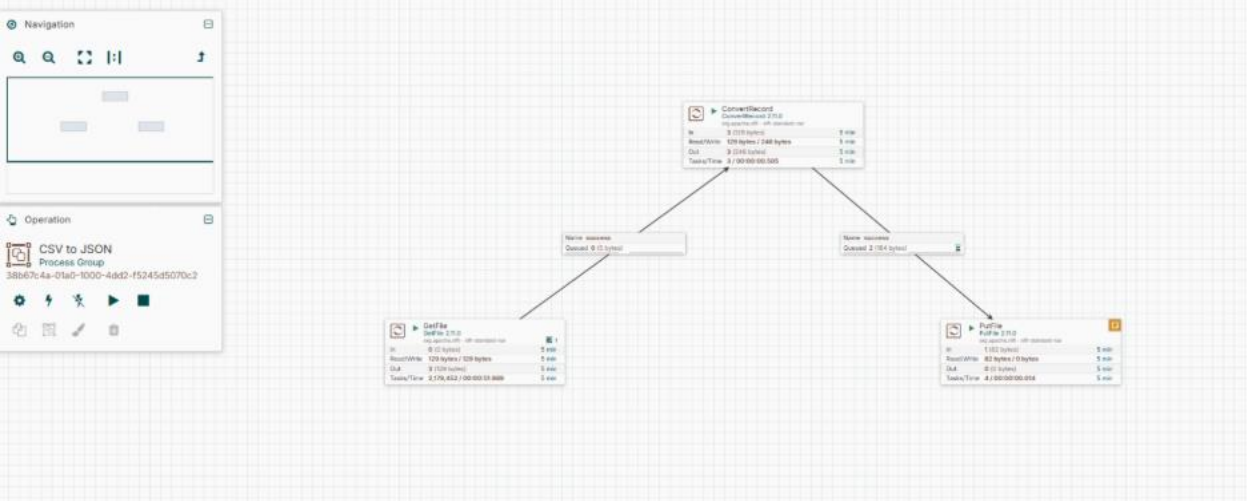
Goal: Introduce Record-based processors and schema handling.

Processors used: `GetFile`, `ConvertRecord` (with `CSVReader` and `JSONRecordSetWriter`), `PutFile`

Steps

- Create a sample CSV file, `people.csv`, in your input folder: name,age,cityAsha,29,MumbaiRahul,34,Pune
    1.
    2. Add `GetFile` pointing to your input folder.
    3. Add `ConvertRecord`. Before configuring it, you need two Controller Services:
        a. Click the arrow icon next to `Record Reader` → **Create New Service** → choose `CSVReader` → **Create**.
        b. Click the arrow icon next to `Record Writer` → **Create New Service** → choose `JSONRecordSetWriter` → **Create**.
    4. Go to the **Controller Services** tab (gear icon at bottom-left of canvas, or top-right menu → **Controller Services**). Find `CSVReader` and `JSONRecordSetWriter`.
    5. Configure `CSVReader`: set `Schema Access Strategy` to `Use String Fields From Header` (this auto-detects columns from the CSV header row).
    6. Configure `JSONRecordSetWriter`: leave defaults, but ensure `Schema Write Strategy` doesn't conflict — default settings work fine for this exercise.
    7. Enable (start) both controller services using the lightning bolt icon next to each.
    8. Back on `ConvertRecord`, confirm `Record Reader` = `CSVReader` and `Record Writer` = `JSONRecordSetWriter`.
    9. Add `PutFile` for your output folder.

10. Connect `GetFile` → `ConvertRecord` (`success`) → `PutFile` (`success`).
11. Start everything and check the output — it should be a JSON array of your CSV rows.



Exercise 5: Routing on Content

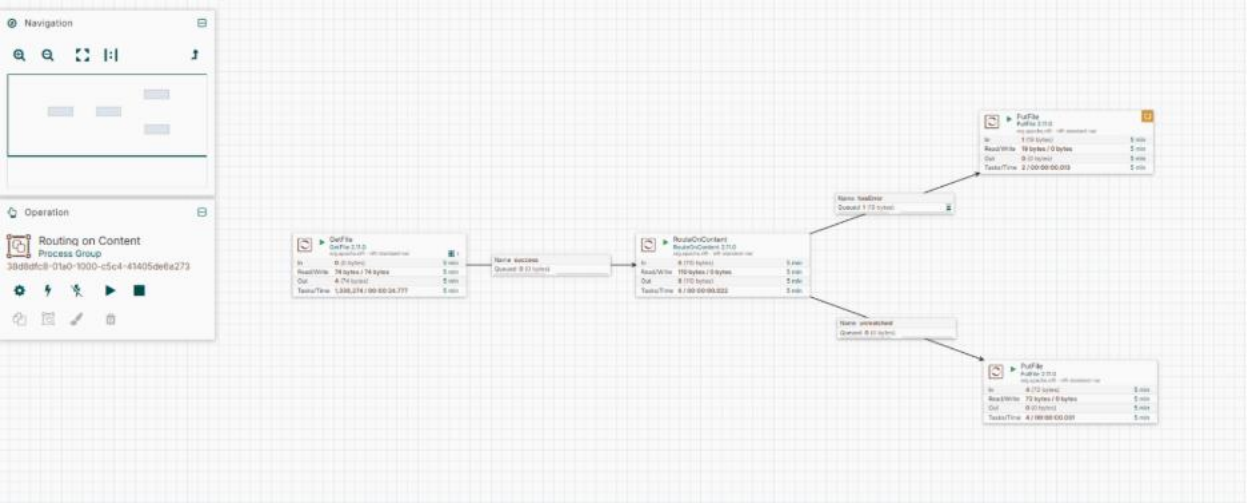
Goal: Learn conditional branching and NiFi relationships.

Processors used: `GetFile`, `RouteOnContent`, two `PutFile` processors

Steps

1. Prepare two sample files in your input folder — one containing the word `ERROR`, one that doesn't.
2. Add `GetFile` pointing to the input folder.
3. Add `RouteOnContent`. Configure:
    a. `Match Requirement`: `content must contain match`
    b. Add a new dynamic property (click `+` on Properties tab): name it `hasError`, value: `ERROR`
4. Add two `PutFile` processors: one pointing to `~/nifi-output/errors`, the other to `~/nifi-output/clean`. Create these folders first.
5. Connect `GetFile` → `RouteOnContent` (`success`).
6. Connect `RouteOnContent` → first `PutFile` on the `hasError` relationship.
7. Connect `RouteOnContent` → second `PutFile` on the `unmatched` relationship.
8. Start all processors and drop both sample files in the input folder.

9. Confirm the error file lands in `errors/` and the clean file lands in `clean/`.



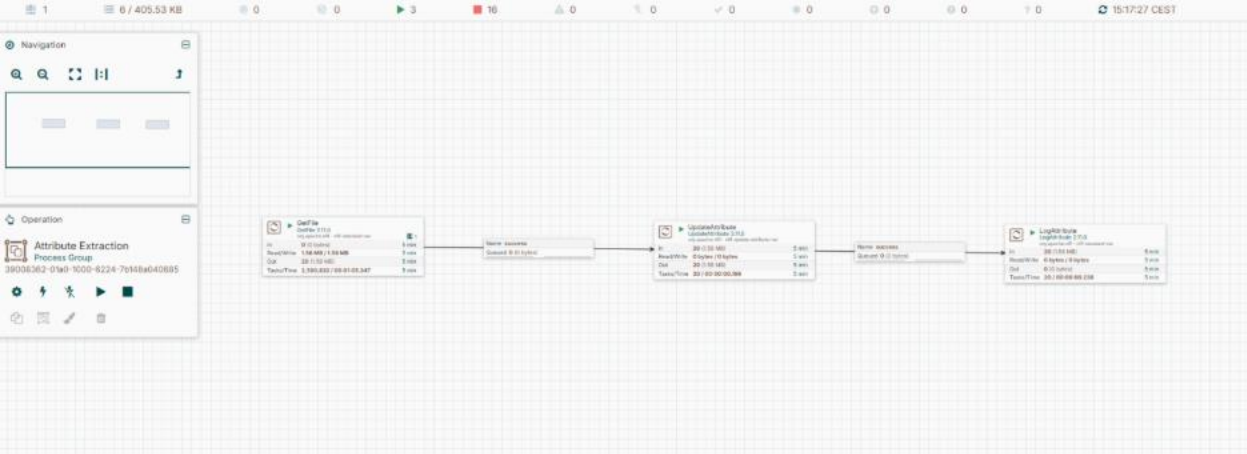
Exercise 6: Attribute Extraction

Goal: Understand the difference between FlowFile content and attributes.

Processors used: `GetFile`, `UpdateAttribute`, `LogAttribute`

Steps

1. Add `GetFile` pointing to your input folder (reuse `sample.txt` from earlier).
2. Add `UpdateAttribute`. Add a new dynamic property:
    a. Name: `source.folder`
    b. Value: `nifi-input`
3. Add another dynamic property using NiFi Expression Language:
    a. Name: `file.sizeInKB`
    b. Value: `${fileSize:divide(1024)}`
4. Add `LogAttribute`. Configure `Log Level` to `info` and `Attributes to Log` to blank (logs all attributes).
5. Connect `GetFile` → `UpdateAttribute` (`success`) → `LogAttribute` (`success`).
6. Start everything and check `nifi-app.log` — you should see `source.folder` and `file.sizeInKB` listed among the FlowFile's attributes, while the *content* of the file remains untouched.



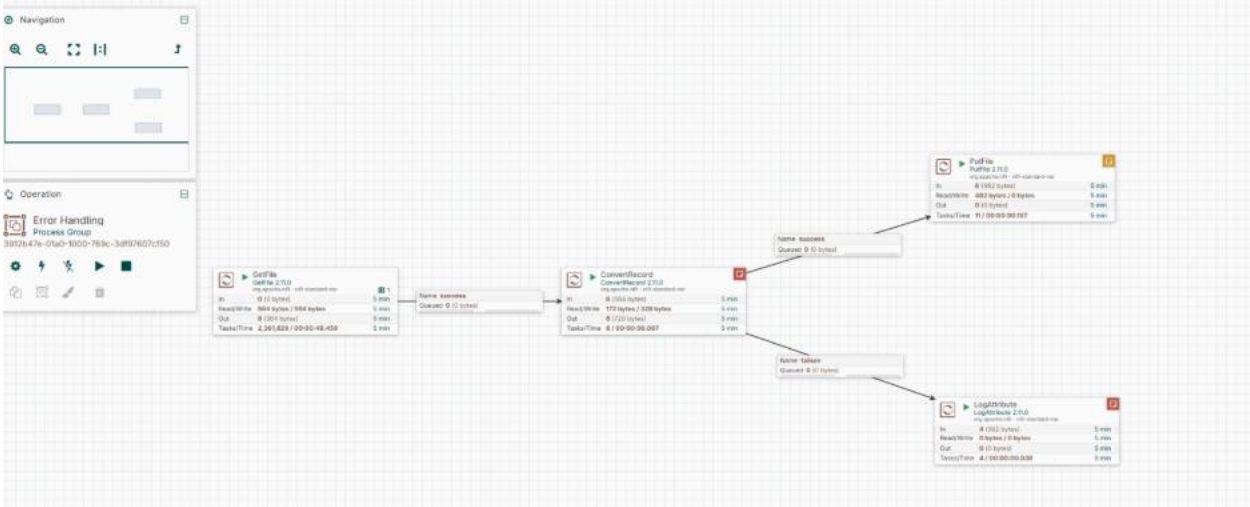
Exercise 7: Error Handling

Goal: Learn NiFi's expectation that every relationship must be handled.

Processors used: Reuse Exercise 2 or 4's flow, plus `LogAttribute` for failures

Steps

1. Open your flow from Exercise 4 (CSV to JSON).
2. Click on the `ConvertRecord` processor — notice the `failure` relationship is likely unset (shown with a warning icon if left unhandled).
3. Add a `LogAttribute` processor and configure `Log Level` to `error`.
4. Connect `ConvertRecord` → this new `LogAttribute` on the `failure` relationship.
5. Start it. To test, drop a malformed CSV file (e.g., mismatched columns or corrupted text) into your input folder.
6. Confirm the failure path logs the issue instead of leaving FlowFiles stuck in a queue with no downstream connection.



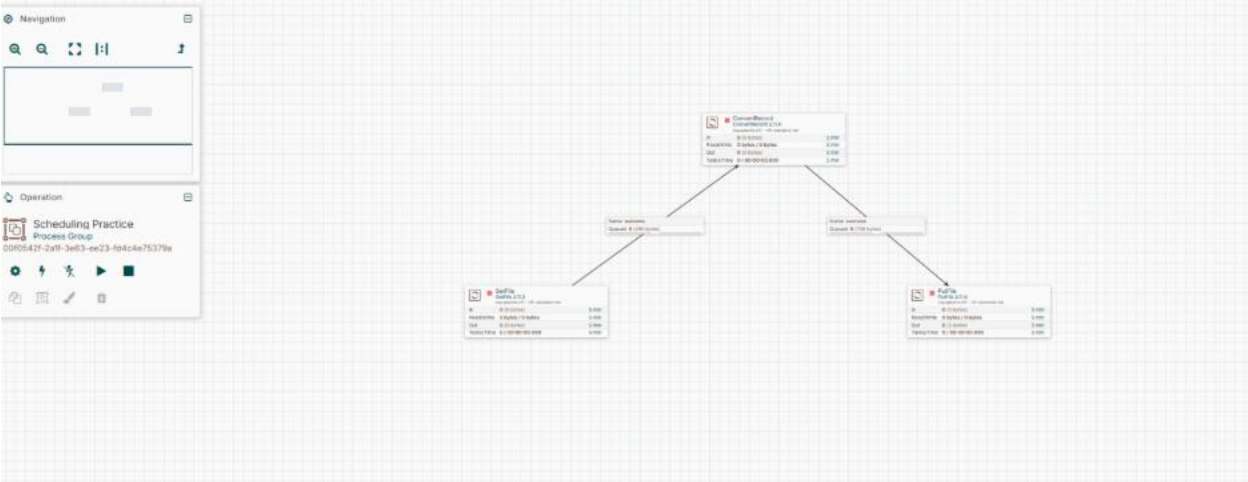
Exercise 8: Scheduling Practice

Goal: Understand processor scheduling strategies.

Processors used: Reuse Exercise 4's `GetFile`

Steps

1. Right-click `GetFile` → **Configure** → **Scheduling** tab.
2. Change `Scheduling Strategy` from `Timer driven` to `CRON driven`.
3. Set the CRON expression to run every minute: `0 * * * * ?`
4. Apply and start the processor. Watch the **Data Provenance** or output folder to confirm it runs on schedule.
5. Switch back to `Timer driven` and set `Run Schedule` to `5 sec`. Compare behavior.



Exercise 9: Simple REST Pull

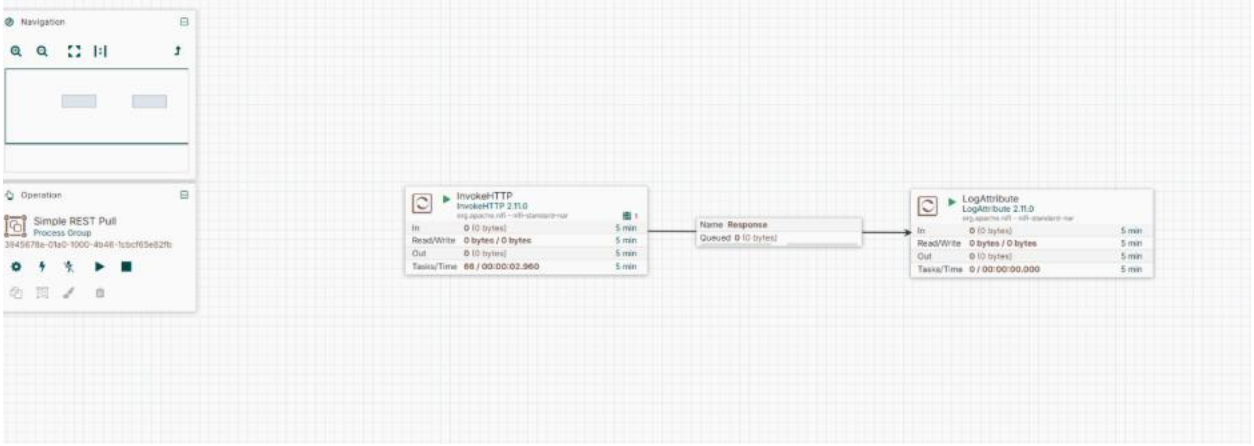
Goal: Introduce HTTP-based ingestion.

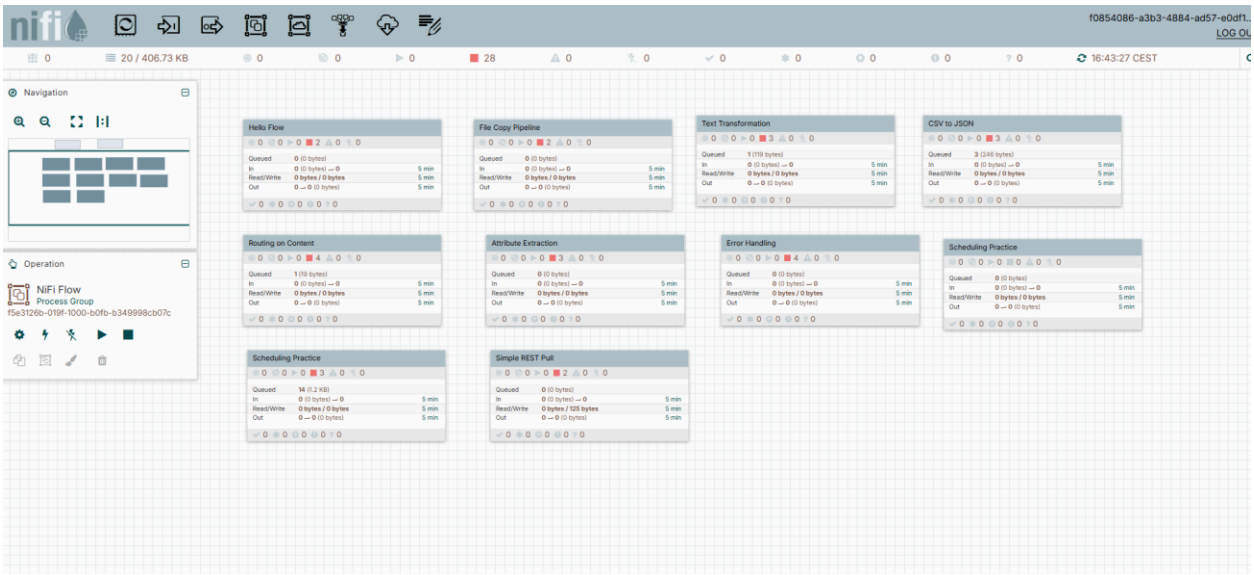
Processors used: `InvokeHTTP`, `LogAttribute` (or `PutFile`)

Steps

1. Add `InvokeHTTP`. Configure:
    a. `HTTP Method`: `GET`
    b. `Remote URL`: https://api.agify.io?name=michael
2. Leave `SSL Context Service` blank (not required for this public HTTPS API in most NiFi versions — if you hit SSL errors, create a `StandardSSLContextService` with defaults and enable it).
3. Add `LogAttribute` and connect `InvokeHTTP` → `LogAttribute` on the `Response` relationship.
4. You'll also need to handle other relationships (`Failure`, `No Retry`, `Retry`) — connect them to a second `LogAttribute` or terminate them (right-click connection → configure as auto-terminated in the processor's Relationships tab).
5. Start the flow and check the log for the JSON response (something like `{"count":..., "name":"michael","age":...}`).

Check yourself: Swap the URL for a different public API (e.g. https://api.coindesk.com/v1/bpi/currentprice.json) and confirm it still works.





Exercise 10: Mini ETL Pipeline (Capstone)

Goal: Combine ingestion, transformation, routing, and egress into one pipeline.

Processors used: `InvokeHTTP`, `EvaluateJsonPath`, `RouteOnAttribute`, `ConvertRecord`, `PutFile`

Steps

1. Add `InvokeHTTP` configured to call https://api.agify.io?name=michael (as in Exercise 9).
2. Add `EvaluateJsonPath`. Set `Destination` to `flowfile-attribute`. Add a dynamic property:
    a. Name: `age`
    b. Value: `$.age`

3. Connect `InvokeHTTP` → `EvaluateJsonPath` on the `Response` relationship.
4. Add `RouteOnAttribute`. Add a dynamic property:
    a. Name: `isAdult`
    b. Value: `${age:ge(18)}`
5. Connect `EvaluateJsonPath` → `RouteOnAttribute` (`matched`).
6. Add `ConvertRecord` with a `JsonTreeReader` and `CSVRecordSetWriter` (create these Controller Services the same way as Exercise 4, then enable them).
7. Connect `RouteOnAttribute` → `ConvertRecord` on the `isAdult` relationship.
8. Add `PutFile` pointing to an output folder.
9. Connect `ConvertRecord` → `PutFile` (`success`).
10. Handle all remaining relationships (failure paths, `unmatched` from `RouteOnAttribute`) by connecting them to a `LogAttribute` or auto-terminating them.
11. Start the entire flow and confirm a CSV file appears in your output folder containing the converted JSON response.

General Tips While Practicing

- **Use `LogAttribute` as your debug tool.** It's the fastest way to see what's actually inside a FlowFile (both content and attributes).
- **Check Data Provenance often.** Right-click any processor → *View Data Provenance* to trace a FlowFile's full history.
- **Inspect queues before troubleshooting.** Right-click a connection → *List Queue* to see FlowFiles waiting between processors.
- **Every relationship must be handled.** NiFi won't let you start a processor with an unconnected relationship unless it's explicitly auto-terminated.
- **Keep exercises in separate Process Groups.** This keeps your canvas clean and makes it easy to revisit earlier exercises.